HANDS-ON LAB REPORT

AWS WAF

Protecting a Login Page

Web Application Firewall on an Application Load Balancer
SQL injection and brute-force protection, tested live



4	4	20
Protection rules	Tests performed	Documented steps

Submitted by	Anjali Singh
Date	03 October 2026
Cloud platform	Amazon Web Services (AWS)
Region	Asia Pacific (Mumbai) - ap-south-1
Protected web link	waf-alb-369645585.ap-south-1.elb.amazonaws.com/login/

Index

1. About AWS WAF . . . 3

2. Lab Details . . . 3

3. Architecture . . . 4

4. Step-by-Step Lab . . . 4

Step 1 | Set a billing alert . . . 4
Step 2 | Launch the EC2 web server . . . 5
Step 3 | Test the login page on the server IP . . . 5
Step 4 | Create the target group . . . 6
Step 5 | Target group created . . . 6
Step 6 | Set up the load balancer listener . . . 7
Step 7 | Load balancer created . . . 7
Step 8 | First test through the load balancer . . . 8
Step 9 | Login page works through the load balancer . . . 8
Step 10 | Start creating the WAF protection pack . . . 9
Step 11 | Protection pack created . . . 9
Step 12 | Add the rate-based rule for /login . . . 10
Step 13 | Rate-based rule settings (JSON view) . . . 10
Step 14 | Test a normal login . . . 11
Step 15 | Test SQL injection . . . 11
Step 16 | Test brute-force protection . . . 12
Step 17 | WAF log: SQL injection blocked . . . 12
Step 18 | WAF log: internet scans blocked . . . 13
Step 19 | WAF log: rate-based rule blocking . . . 13
Step 20 | WAF log: normal requests allowed . . . 14

5. Test Results . . . 14

1. About AWS WAF

AWS WAF (Web Application Firewall) checks every web request that comes to an application and decides whether to allow it or block it. It works at the web (HTTP/HTTPS) level, so it can stop attacks such as SQL injection, harmful scans and too many login attempts before they reach the server.

WAF can be attached to an Application Load Balancer, CloudFront, API Gateway and other AWS services. In this lab it is attached to an Application Load Balancer that sits in front of a demo login page.

Key terms used in this report

Protection pack (web ACL)	The main container that holds all the rules. Here it is named **login-waf-acl**.
Rule	A condition with an action, such as Allow or Block.
Managed rule group	A ready-made set of rules maintained by AWS (for example the Core rule set and the SQL injection rules).
Rate-based rule	Counts requests from each IP address and blocks the IP when it sends too many.
Sampled requests	A WAF screen that shows example requests, with the rule that matched and the action taken.

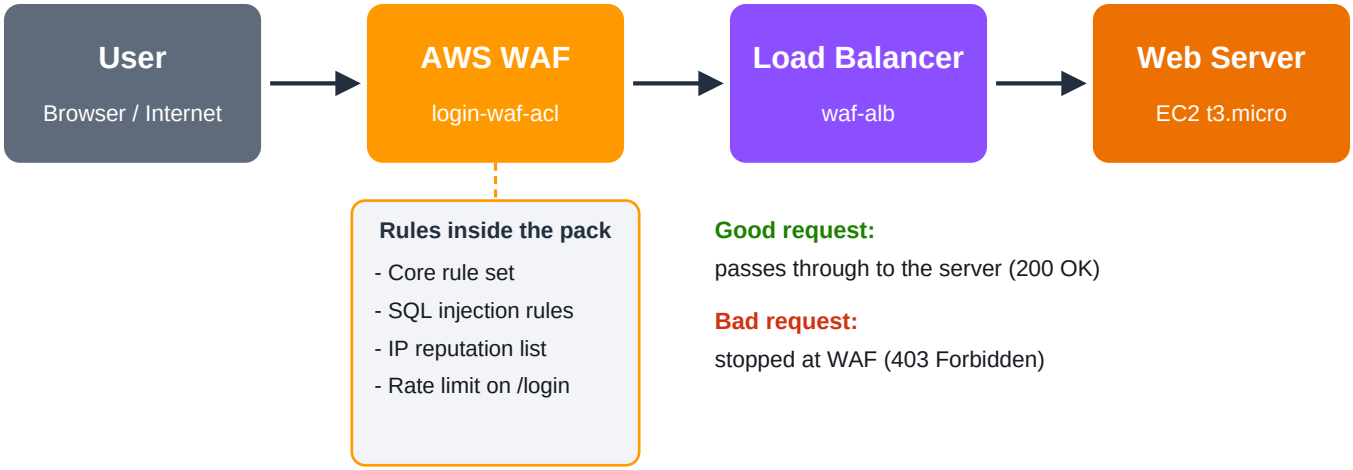
Objective of this lab

To build a demo login page on AWS, protect it with AWS WAF, and prove with tests that normal users are allowed, SQL injection is blocked and repeated login requests are blocked.

2. Lab Details

AWS account ID	573745863981
AWS region	Asia Pacific (Mumbai) - ap-south-1
Web server public IP	13.204.43.135 (EC2 instance, t3.micro, Amazon Linux 2023)
Domain link (load balancer)	waf-alb-369645585.ap-south-1.elb.amazonaws.com
Login page link	http://waf-alb-369645585.ap-south-1.elb.amazonaws.com/login/
Load balancer	waf-alb (Application Load Balancer, internet-facing, zones ap-south-1a and ap-south-1b)
Target group	waf-tg (HTTP port 80, health check path /login/)
VPC	vpc-07a05be83cb2c4e87 (default VPC)
WAF protection pack	login-waf-acl (scope: Regional)
Rules applied	AWS Core rule set, AWS SQL database rule set, Amazon IP reputation list, custom Rate-based_rule
Rate-limit setting	10 requests per IP address in 5 minutes, only for URLs containing /login, action Block
Test machine IP (seen in WAF)	49.36.139.204 (India)

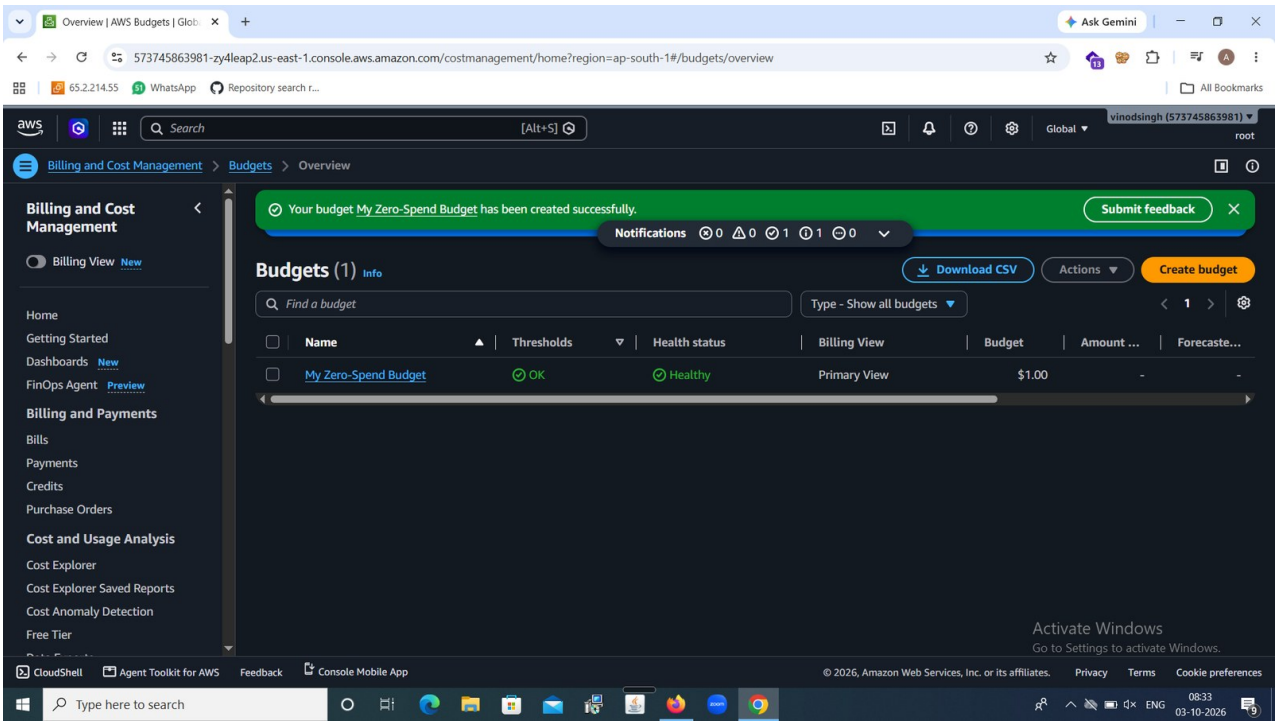
3. Architecture



A visitor's request first goes to AWS WAF. WAF checks it against the rules. Good requests continue to the load balancer and then to the web server. Bad requests are stopped at WAF, and the visitor sees a 403 Forbidden page.

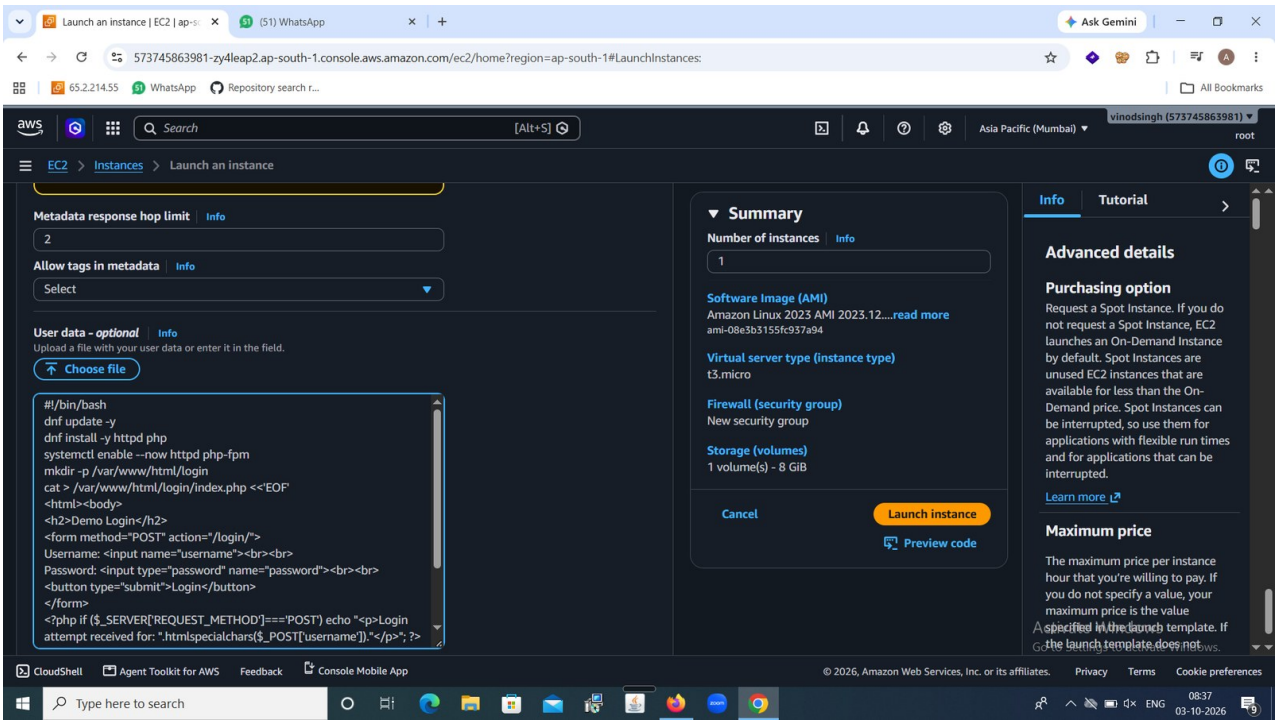
4. Step-by-Step Lab

Step 1 | Set a billing alert



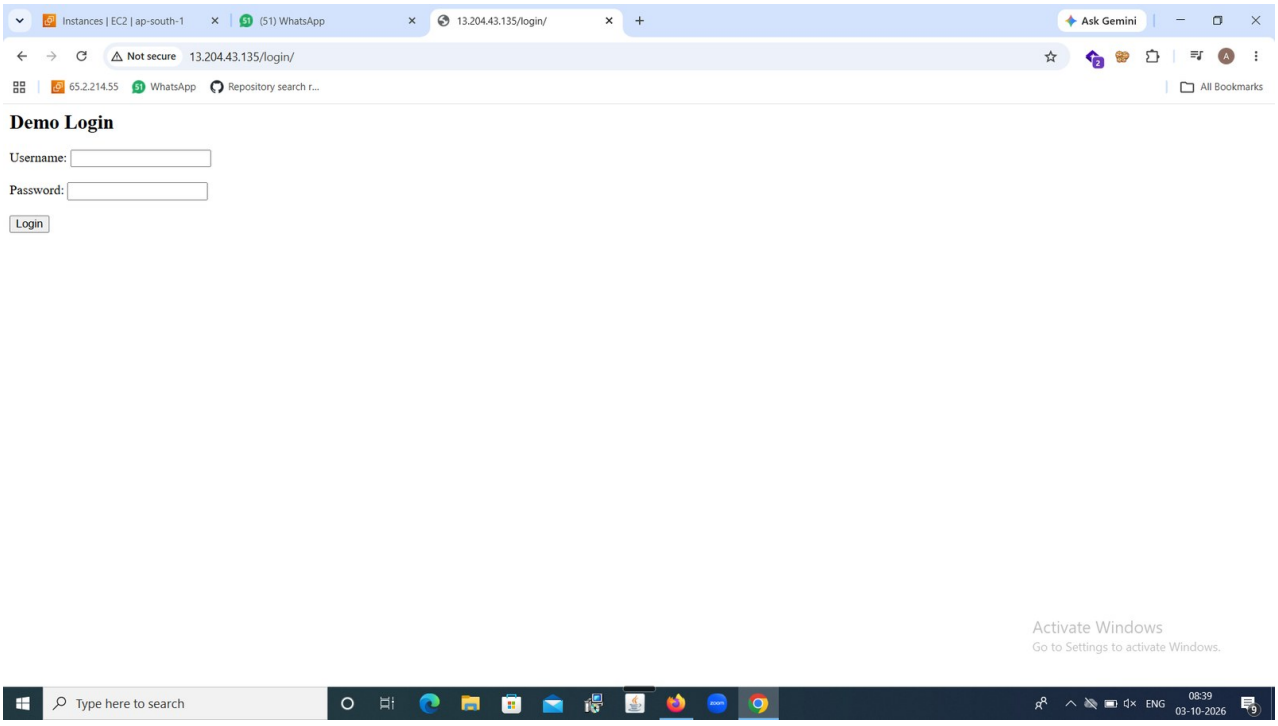
A zero-spend budget was created first, so AWS sends an alert if any charge appears. The screenshot shows **My Zero-Spend Budget** with status Healthy and a limit of $1.00. This keeps the account safe while testing services that can cost money, such as the load balancer and WAF.

Step 2 | Launch the EC2 web server



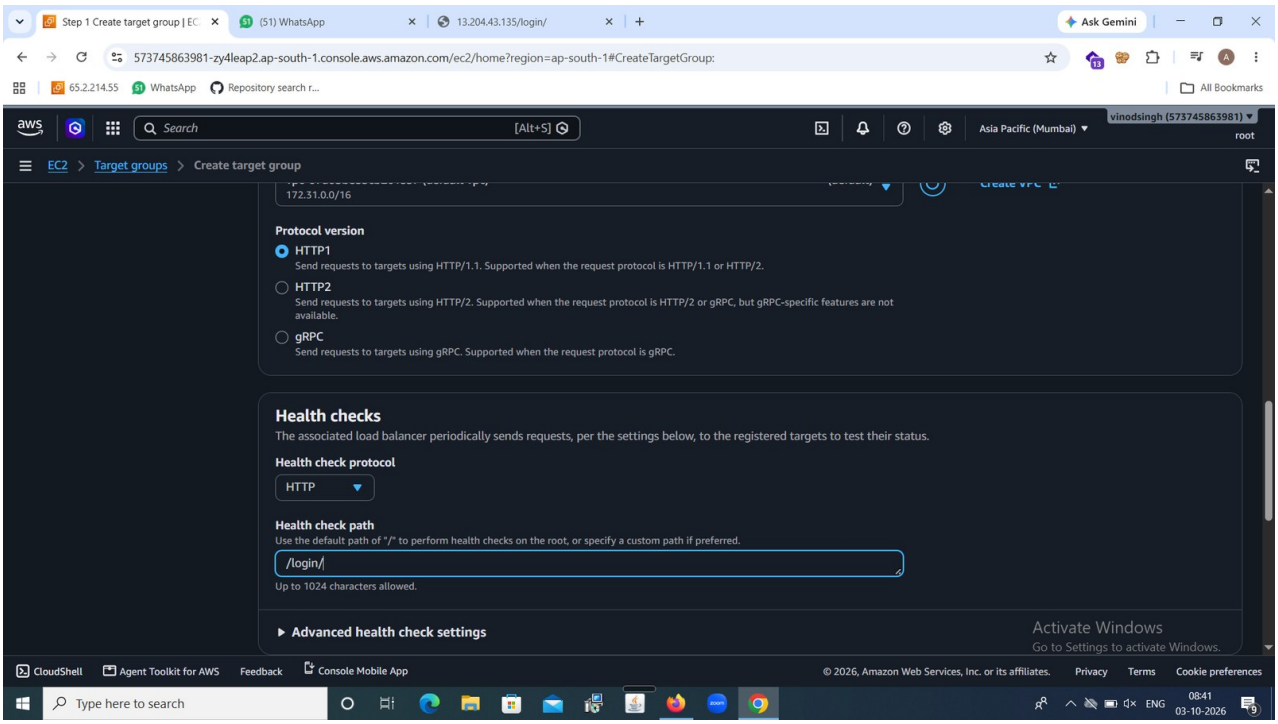
An EC2 server was launched in the Mumbai region using **Amazon Linux 2023** and the **t3.micro** size. A short start-up script (user data) installs Apache and PHP and creates a simple **Demo Login** page at /login/. This is the application that WAF will protect.

Step 3 | Test the login page on the server IP



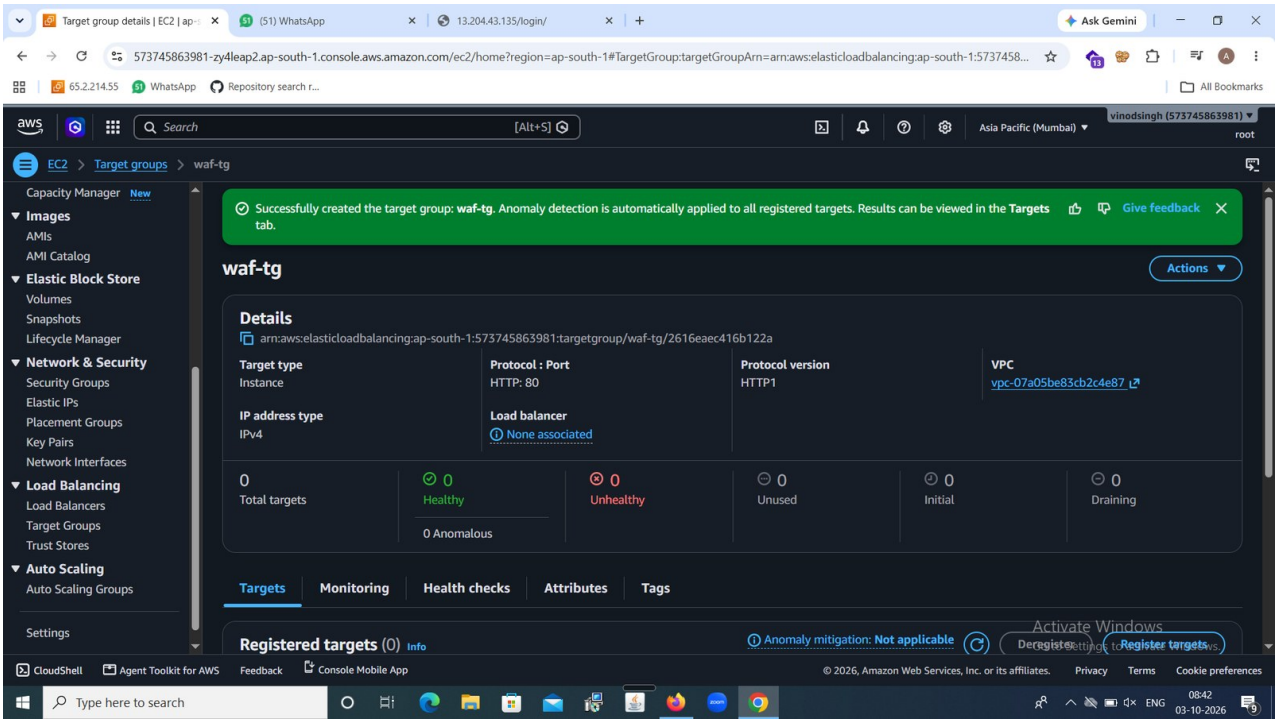
The login page was opened with the server's public IP address, **13.204.43.135/login/**. The Demo Login form with Username, Password and a Login button loads correctly. This confirms the web server works before the load balancer and WAF are added.

Step 4 | Create the target group



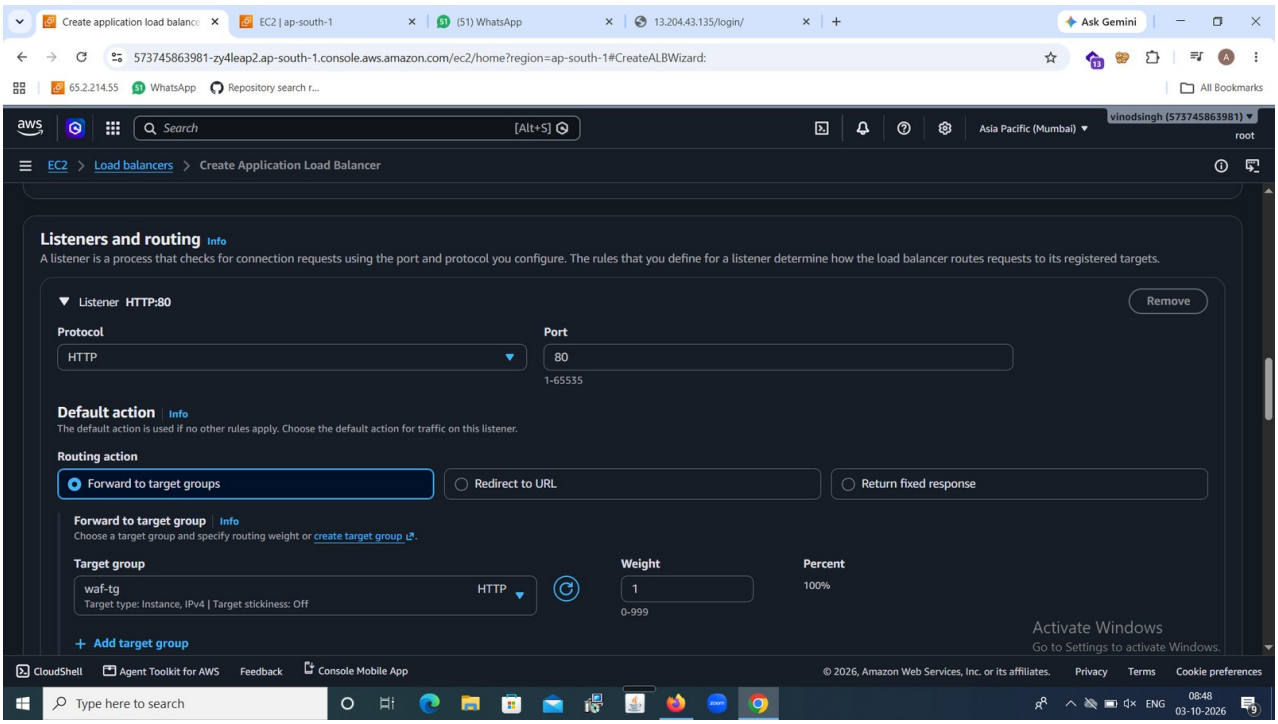
A target group named **waf-tg** was created for the load balancer. The health check path was set to **/login/**, so the load balancer checks the login page to know if the server is healthy. The protocol is HTTP on port 80.

Step 5 | Target group created



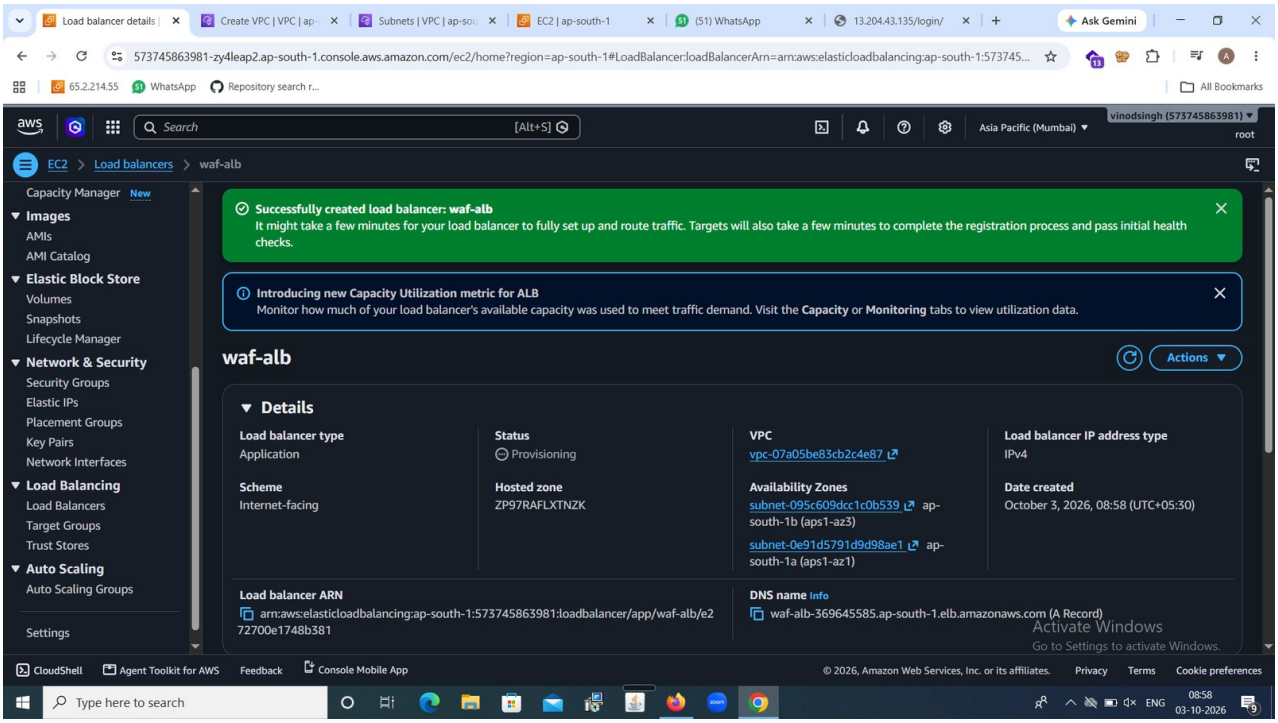
The target group **waf-tg** was created successfully in the default VPC. At this point it shows 0 registered targets, which means no server is attached yet. The EC2 server is registered to it in the next steps.

Step 6 | Set up the load balancer listener



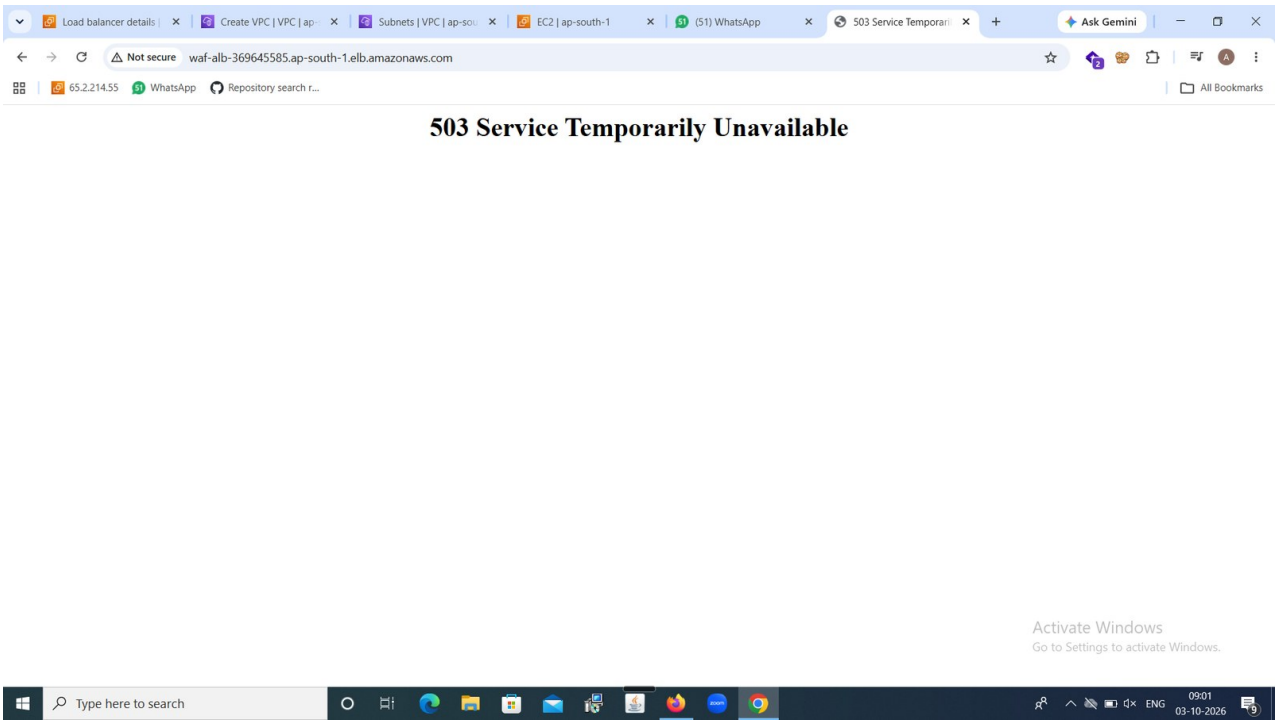
The Application Load Balancer was configured with a listener on **HTTP port 80**. The listener forwards all traffic to the target group **waf-tg** (100 percent). Users will now reach the server only through the load balancer.

Step 7 | Load balancer created



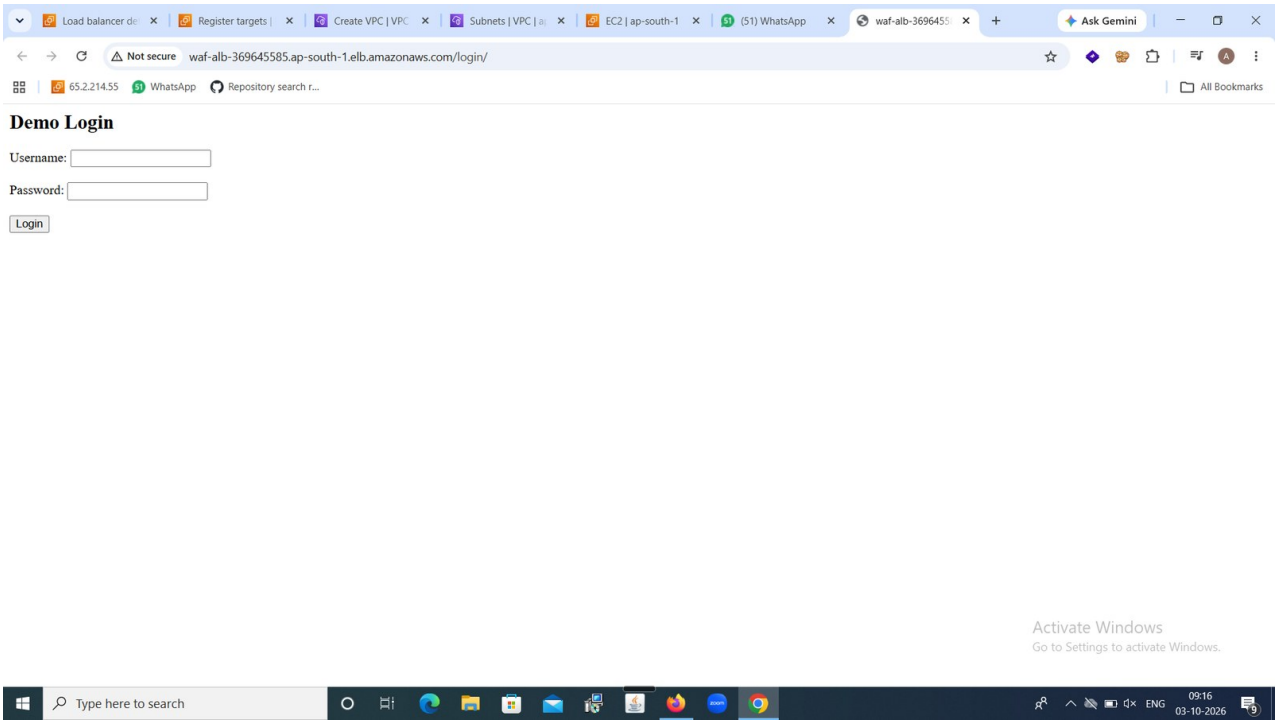
The load balancer **waf-alb** was created. It is internet-facing, uses two zones (ap-south-1a and ap-south-1b) and first shows the status Provisioning. Its DNS name, **waf-alb-369645585.ap-south-1.elb.amazonaws.com**, is the web link used for all later tests.

Step 8 | First test through the load balancer



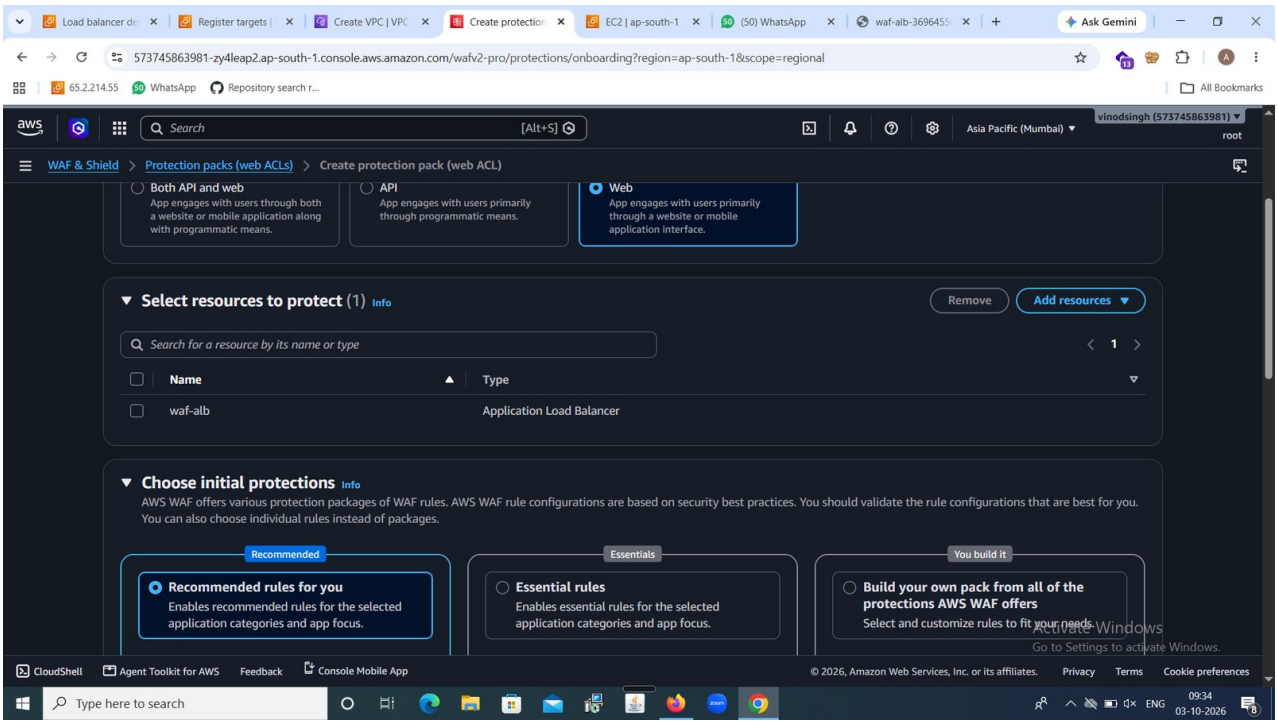
The first test of the load balancer link showed **503 Service Temporarily Unavailable**. This happened because the EC2 server was not yet registered in the target group, so the load balancer had nowhere to send traffic. The server was then registered as a target.

Step 9 | Login page works through the load balancer



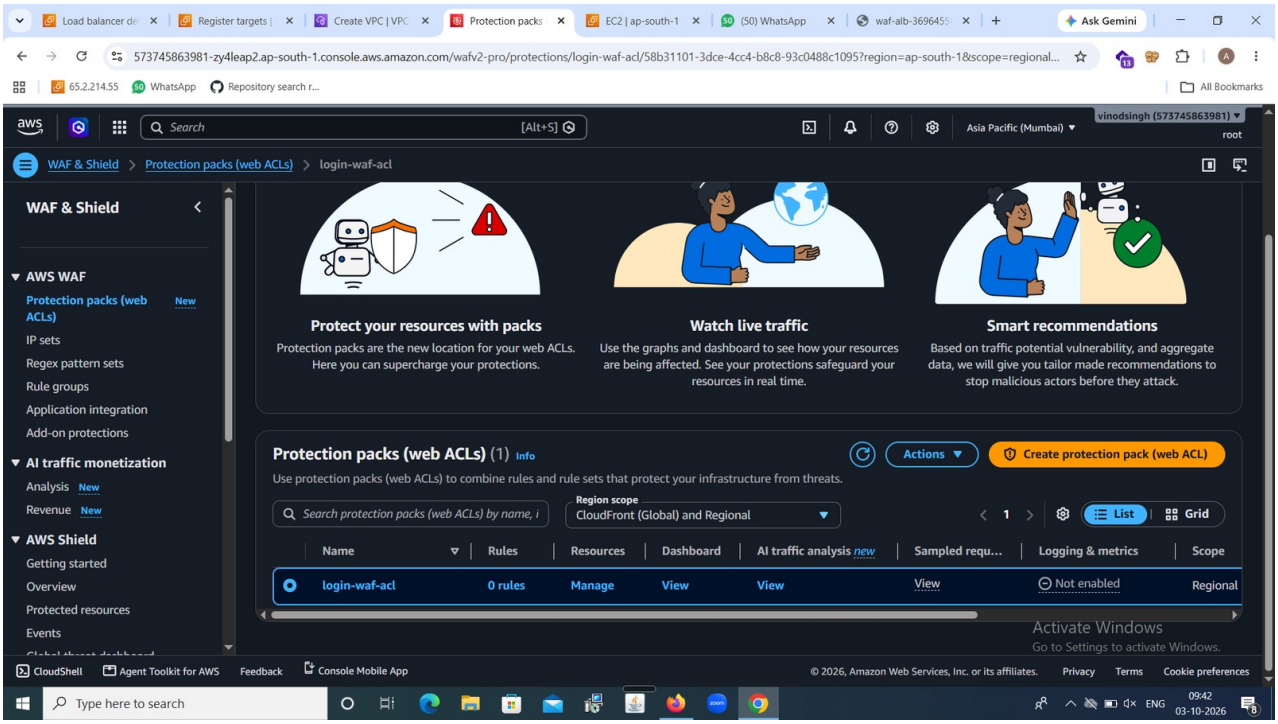
After the server was registered and became healthy, the login page opened through the load balancer link. The address bar shows **waf-alb-369645585.ap-south-1.elb.amazonaws.com/login/**. The application is now reachable through the load balancer, ready for WAF protection.

Step 10 | Start creating the WAF protection pack



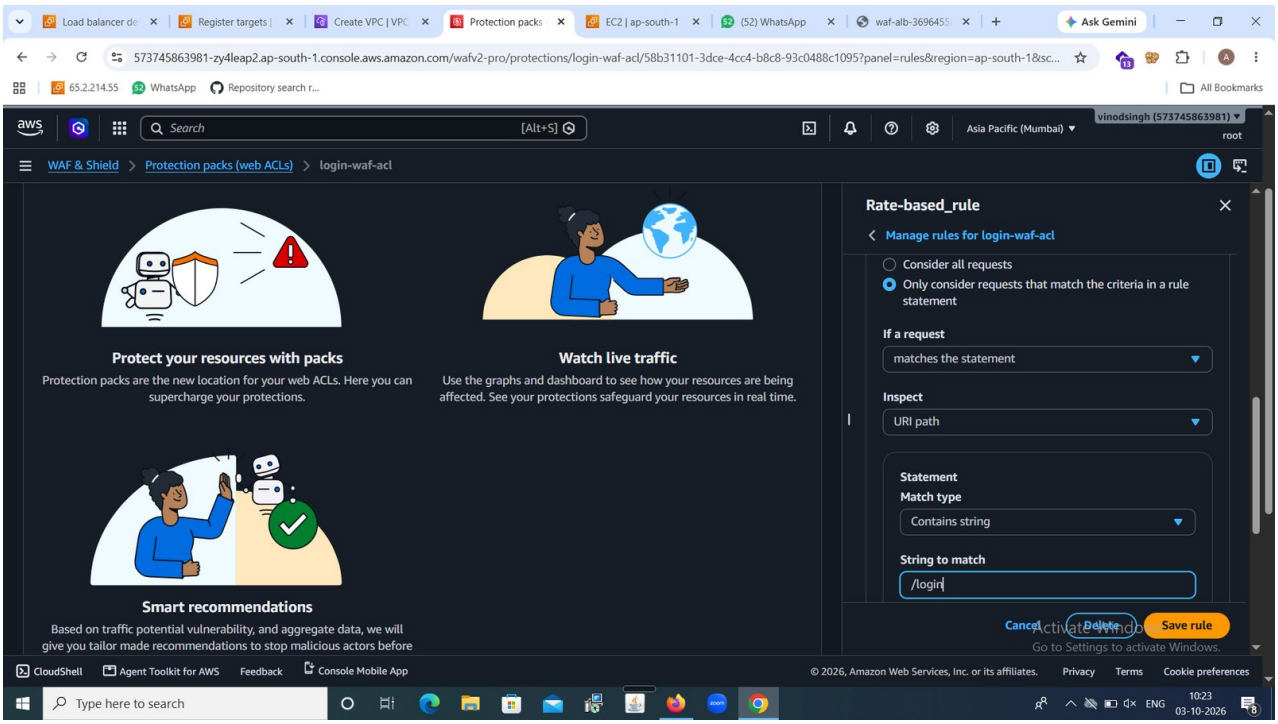
In AWS WAF, a new protection pack (web ACL) was started. The app focus is **Web** and the load balancer **waf-alb** is listed as the resource to protect. Of the three starting options shown, **Build your own pack** was used, so only the needed rules are added and the cost stays low.

Step 11 | Protection pack created



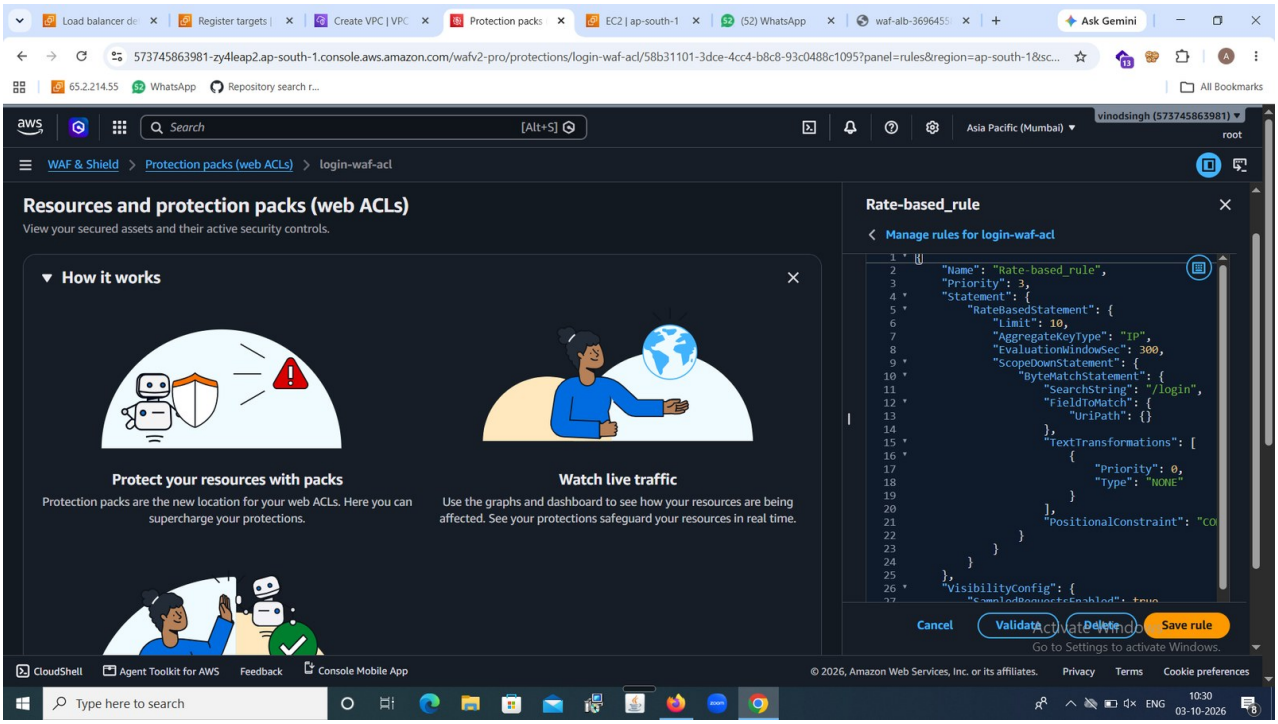
The protection pack **login-waf-acl** was created with scope Regional in the Mumbai region. It first shows 0 rules. Next, three AWS managed rule groups (Core rule set, SQL database rules, IP reputation list) and one custom rate-based rule were added.

Step 12 | Add the rate-based rule for /login



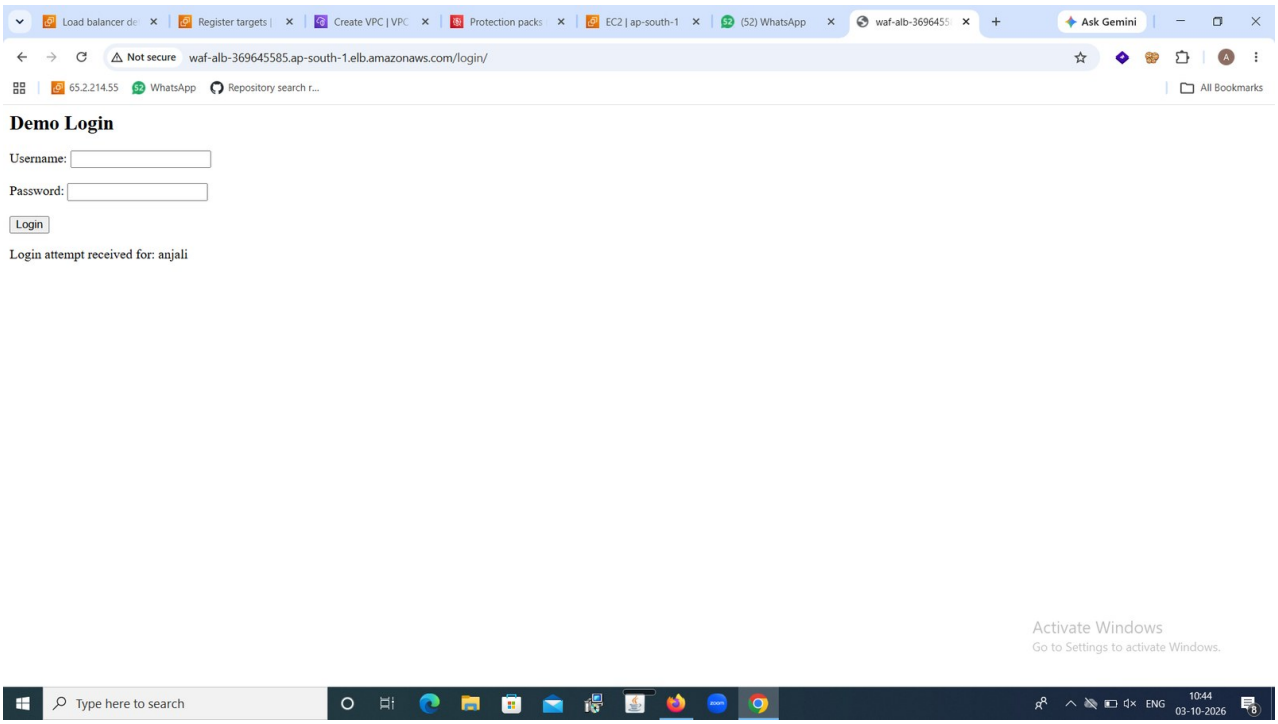
A custom **rate-based rule** was added to stop repeated login attempts. The option 'Only consider requests that match the criteria' was selected. The condition inspects the **URI path** with 'Contains string' and the text **/login**, so only login requests are counted.

Step 13 | Rate-based rule settings (JSON view)



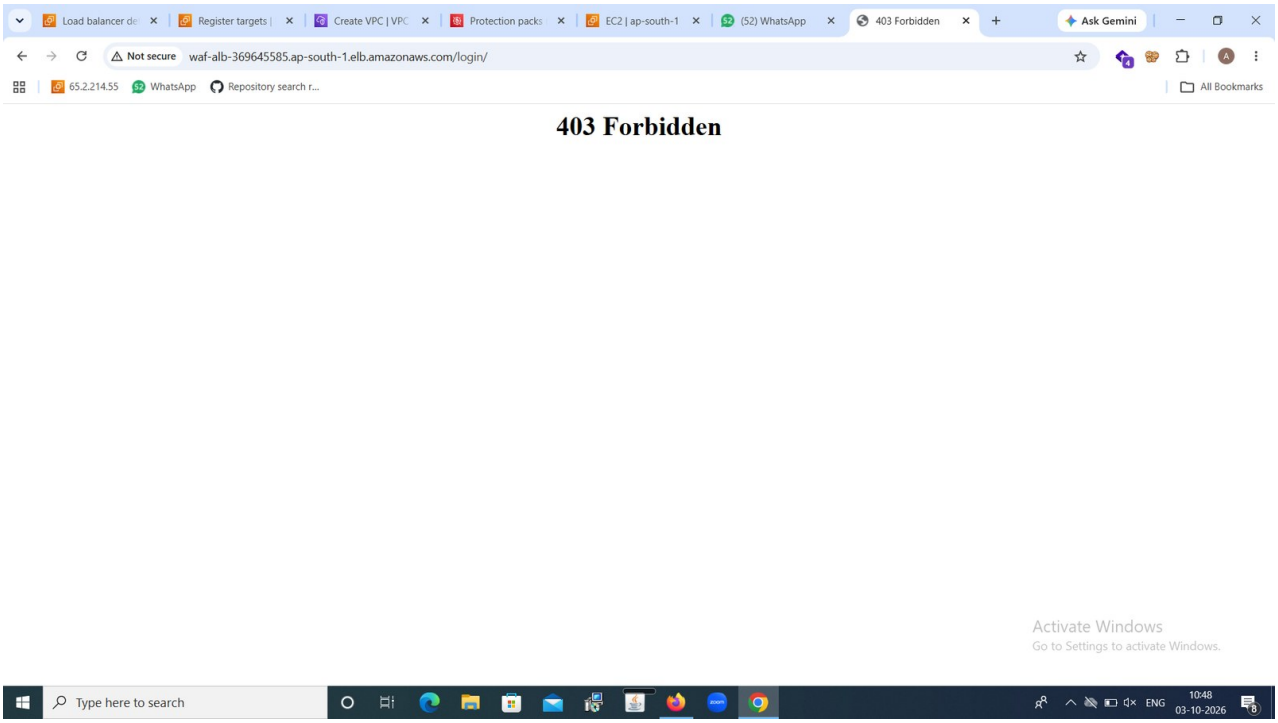
The JSON view confirms the rule: **Limit 10** requests per IP address within **300 seconds** (5 minutes), counted only when the URI path contains **/login**. When an IP goes over this limit, the rule blocks its requests.

Step 14 | Test a normal login



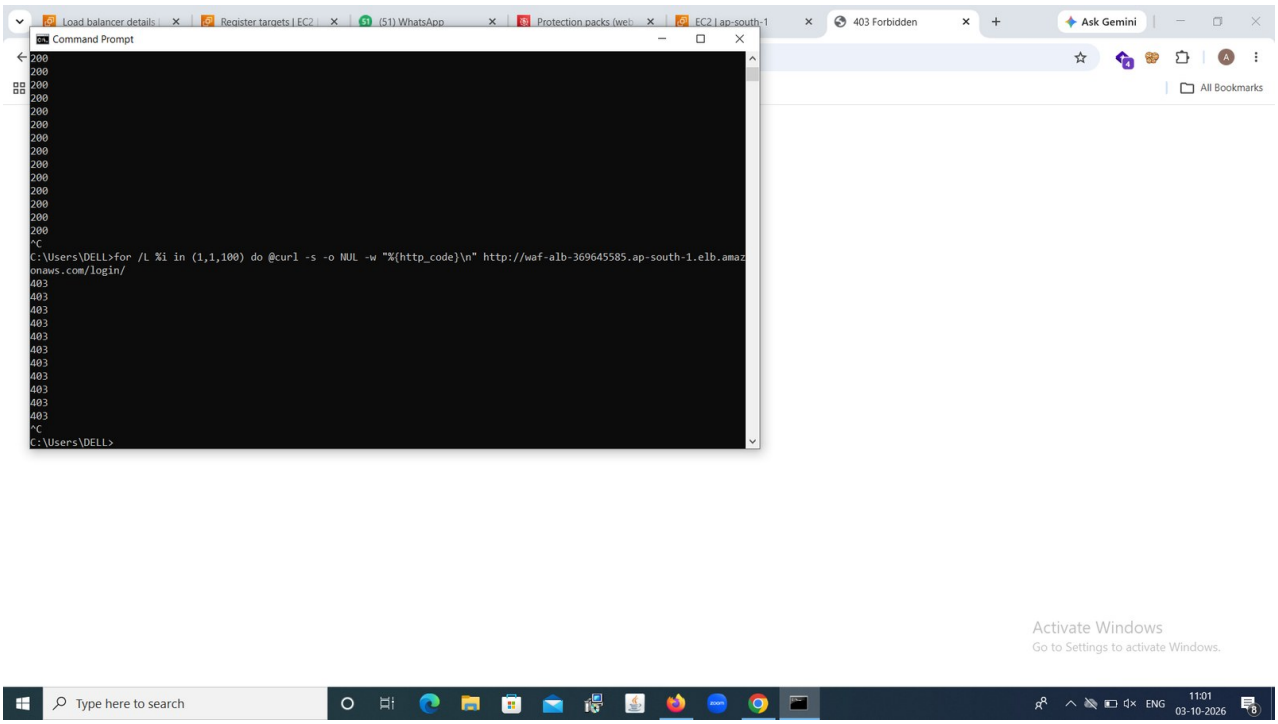
A normal login was tested with the username **anjali**. The page replied 'Login attempt received for: anjali'. This shows that WAF lets genuine users through and normal traffic is not disturbed.

Step 15 | Test SQL injection



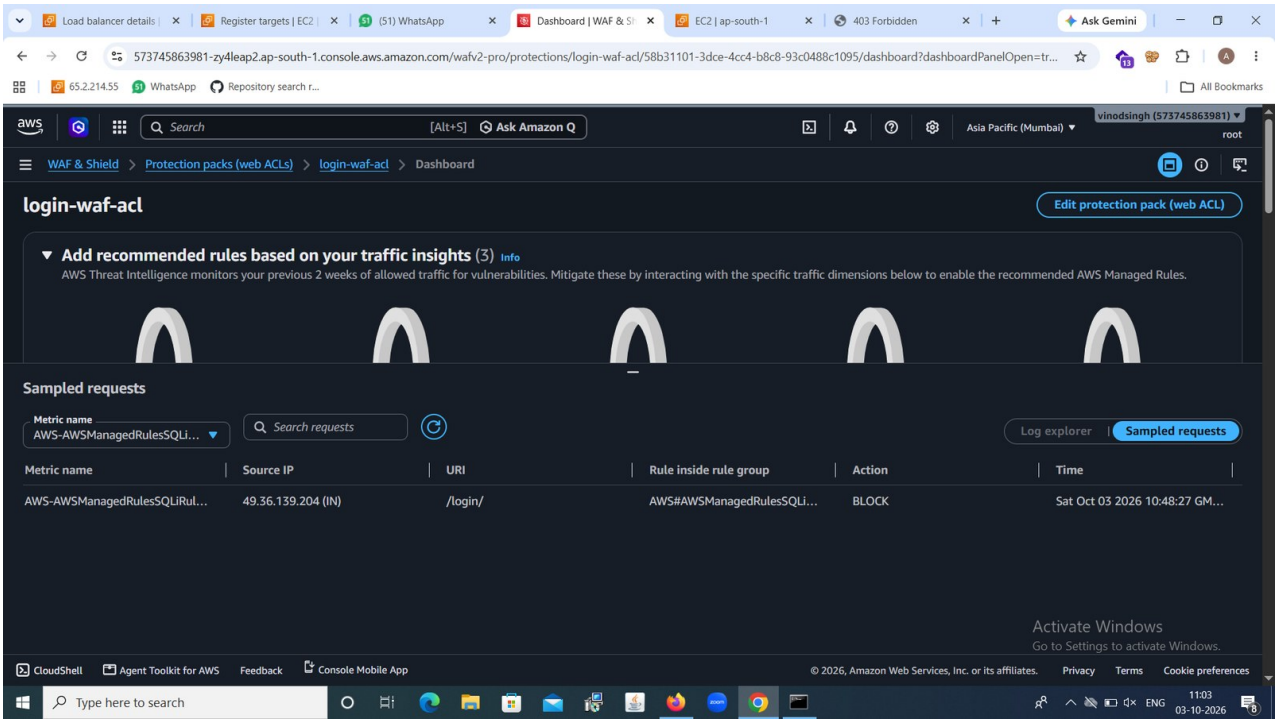
A SQL injection test was done by typing SQL attack text into the login form. WAF stopped the request, and the browser showed **403 Forbidden** on the same load balancer link. The attack never reached the web server.

Step 16 | Test brute-force protection



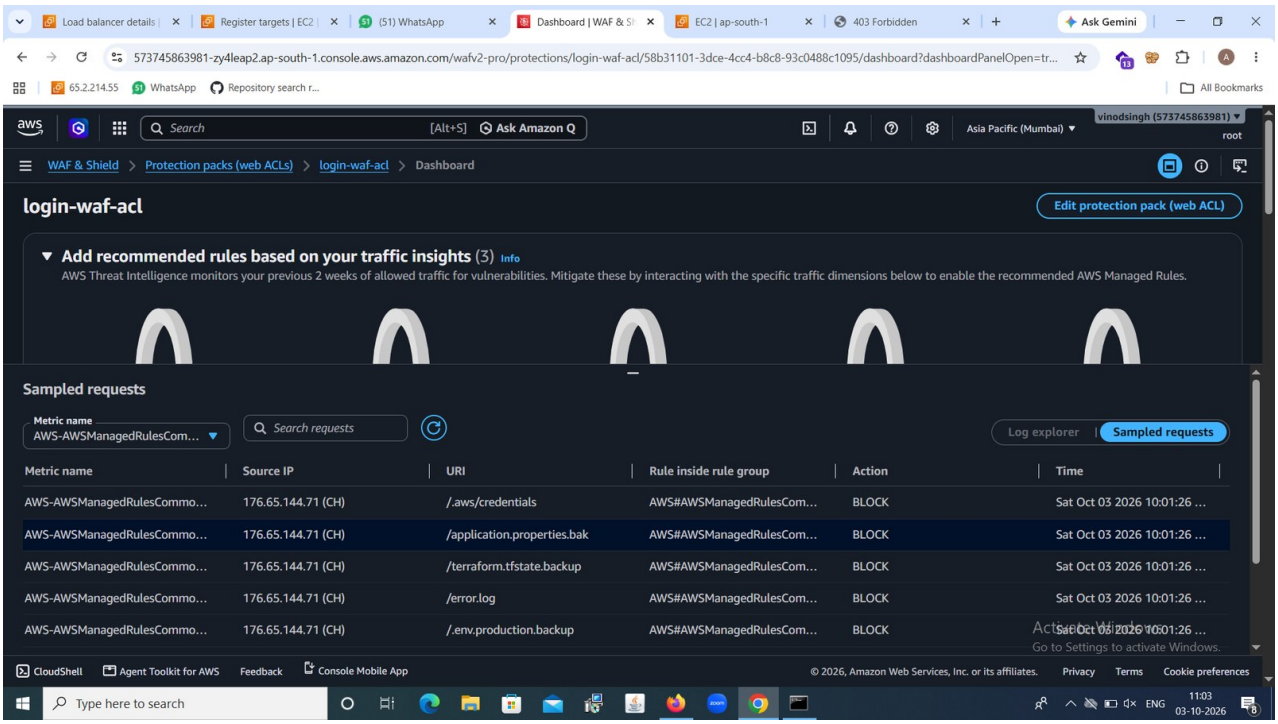
A brute-force style test was run from Command Prompt, using curl to send many requests to /login/. The first requests returned code **200** (allowed). After the limit was crossed, every request returned code **403** (blocked).

Step 17 | WAF log: SQL injection blocked



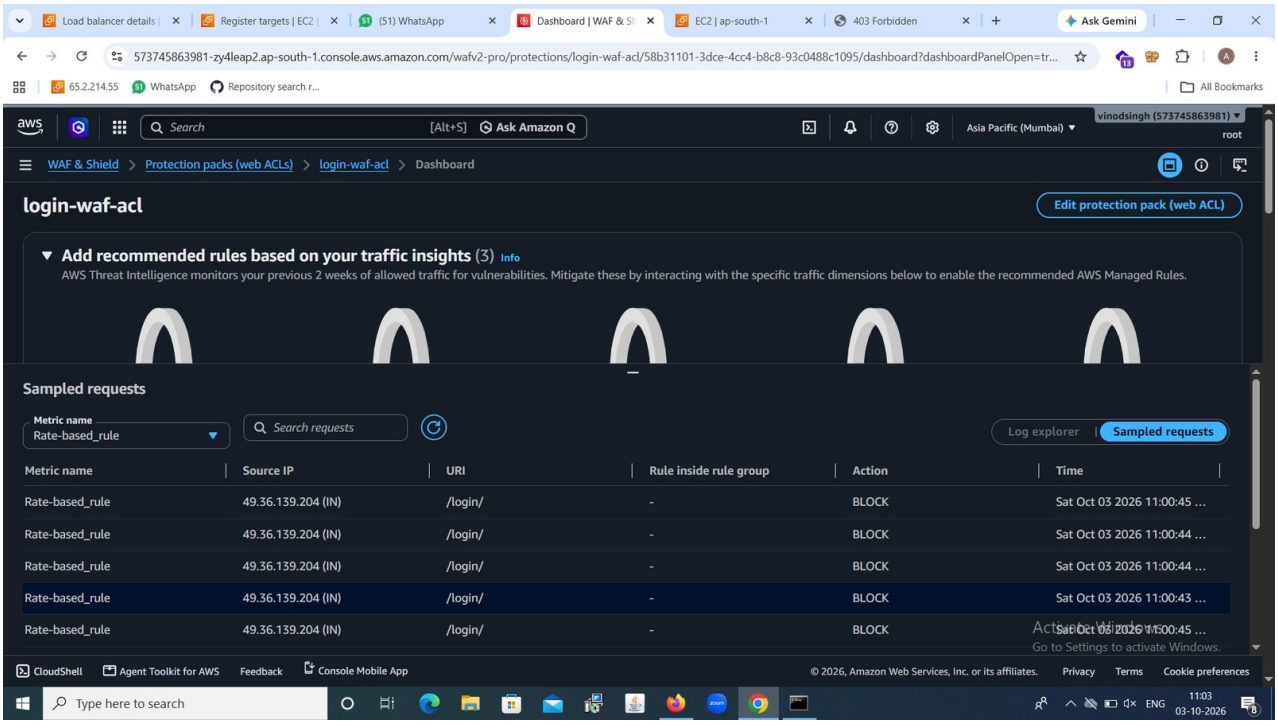
WAF sampled requests show the SQL injection attempt being blocked. The rule group **AWSManagedRulesSQLiRuleSet** matched, the URI is /login/, the action is **BLOCK**, and the source IP is 49.36.139.204 (India) at 10:48 on 03 Oct 2026.

Step 18 | WAF log: internet scans blocked



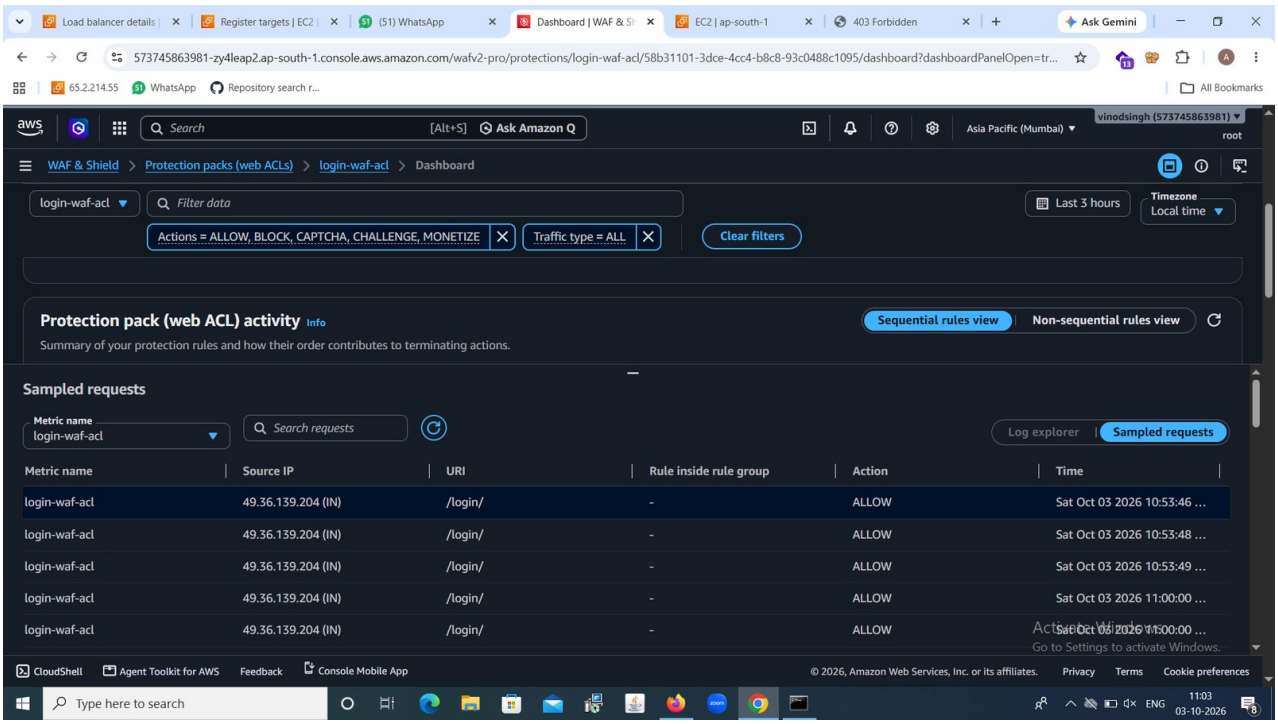
Real scans from the internet also reached the load balancer and were blocked. Source IP **176.65.144.71** (Switzerland) tried to open sensitive files such as /.aws/credentials and /terraform.tfstate.backup. The **Core rule set** blocked these requests (action BLOCK).

Step 19 | WAF log: rate-based rule blocking



The **Rate-based_rule** blocked repeated requests to /login/. Several BLOCK entries from source IP 49.36.139.204 appear within a few seconds, around 11:00:43 to 11:00:45. This matches the Command Prompt test, where the 403 codes started.

Step 20 | WAF log: normal requests allowed



The activity for **login-waf-acl** also lists requests to /login/ with the action **ALLOW**, made at 10:53 and 11:00 before the rate limit was reached. This shows normal use is allowed and only the extra requests are blocked.

5. Test Results

Test	What was done	Result	Rule that acted
Normal login	Logged in with username 'anjali'	**ALLOWED (200)**	None - request passed
SQL injection	SQL attack text entered in the login form	**BLOCKED (403)**	AWSManagedRulesSQLiRuleSet
Too many login requests	Many curl requests sent to /login/	**BLOCKED (403)**	Rate-based_rule
Internet scan probes	Automatic scans for files like /.aws/credentials	**BLOCKED (403)**	AWSManagedRulesCommonRuleSet